Lesson 13 Sidebars

A Hit That Reacts

VIDEO GAME CREATION

These aren't part of the lesson's required work. Think of them the way a textbook sidebar works: a deeper, tangential look at something this lesson's build actually ran into - why one approach was chosen over the others, what the alternatives were, or an idea worth a longer look than the Handout has room for. Pull one out for a discussion moment if it fits, hand it to a curious student, or skip it entirely - nothing here is required reading.

The White Flash That Wasn't (Part Two of the Color Story)

Lesson 2's color sidebar explained why tinting a colored sprite comes out muddy. This lesson's hit-flash effect ran into the exact same root cause - except this time, the result wasn't muddy. It was invisible.

A color tint is a multiply operation: result = original color × tint color, channel by channel. Multiplying by WHITE specifically leaves any color completely unchanged, since multiplying by 1 (white, in normalized color terms) is a no-op. So tinting the enemy white for a hit-flash doesn't just come out muddy on a colored enemy sprite the way Lesson 2's Recolor did - it produces zero visible change at all, since the enemy sprite wasn't white to begin with.

That wasn't hypothetical here: three of eight engines' Level 1 solutions were originally written to set the enemy's tint to white on hit (Godot's modulate, Unity's spriteRenderer.color, GameMaker's image_blend) - all three would have silently done nothing. Easy to miss during a quick test, too, since the enemy still gets destroyed a moment later either way; the missing flash doesn't look obviously broken, it just isn't there.

The fix is the same fix as Lesson 2's Recolor: swap in a genuinely white pre-made sprite instead of tinting toward white. TIC-80's flash was never affected, because it was already built as a sprite-swap from the very start - drawing a dedicated white "flash" version of the enemy directly in the Sprite Editor - the same pattern that made TIC-80 the one engine that never needed Lesson 2's Recolor fix either.

The broader point worth sitting with: this is exactly why "why does this work the way it works" matters more than "does this look right today." The same underlying limitation - a multiply-tint's total dependence on the source art's own color - surfaced twice, in two different lessons, wearing two different-looking symptoms (muddy color, then no visible change at all), because it was the same rule both times. Actually understanding the rule the first time means being able to predict where else it's going to bite - which is a big part of what separates debugging by understanding from debugging by guessing.

What Is an Alarm, Actually? (And Why "6" Isn't Seconds)

obj_bullet's Collision event ends with one quiet line - other.alarm[0] = 6; - that's easy to read past without really registering what it's doing, or what that 6 actually means.

Every single instance in GameMaker - not every object, every individual instance - comes with 12 built-in countdown timers, numbered alarm[0] through alarm[11]. Each one is wired to its own matching Alarm Event: setting alarm[0] is paired with an "Alarm 0" event, alarm[1] with an "Alarm 1" event, and so on. Writing other.alarm[0] = 6; does exactly one thing: it tells that specific enemy instance to start counting down from 6, automatically, with no further code needed to make the countdown happen.

The number that trips people up: 6 is measured in STEPS, not seconds. A step is one frame of the game running at whatever speed the room is set to - commonly 60 steps per second, but that's a project setting, not a fixed rule. GameMaker automatically subtracts 1 from every running alarm each step, checking right after that instance's Begin Step event and right before its regular Step event. The instant an alarm's value hits exactly 0, GameMaker runs that same instance's matching Alarm Event that same step - here, just instance_destroy(). So "6" really means "6 frames from now," and how long that actually takes in real time depends entirely on the room's step speed. At a common 60-steps-per-second room, 6 steps works out to about 0.1 seconds - which is presumably why 6 was the number chosen here: it roughly matches the same 0.1-second pause Godot's await and Unity's yield return use for the identical hit-flash delay. Change the room's speed in a student's own project, though, and the same alarm[0] = 6 line produces a visibly different real-world delay - worth knowing before troubleshooting "my flash looks too fast/slow" as a code problem when it might just be a room-speed difference.

Scope matters too: other.alarm[0] = 6; sets the countdown on the ENEMY specifically (other, inside obj_bullet's Collision-with-obj_enemy event), not on every enemy in the room, and not on the bullet that's running this code. Each instance's 12 alarms belong only to that instance - one enemy's countdown has no effect on any other enemy's.

Worth being explicit about what an alarm ISN'T, since it's an easy thing to assume by analogy with Godot's await or Unity's yield return: those genuinely pause the function they're written inside, mid-execution, and pick back up later in that same function. An alarm does nothing of the kind - setting alarm[0] doesn't pause anything; it just starts an independent countdown running alongside everything else, which later triggers a COMPLETELY SEPARATE event. That's exactly why GameMaker's version of this lesson's idea needs two separate events (the Collision event that starts the flash and sets the alarm, and a new Alarm 0 event that finishes the job) instead of one function that pauses partway through - GML genuinely has no way to pause a single event mid-execution the way those other languages do.

- **A detail worth knowing on sight:** an alarm doesn't just stop at 0 - it keeps counting down to -1 and sits there, which is GameMaker's "not currently running" state for that alarm. That's why the idiomatic way to ask "is this alarm active right now?" is checking alarm[0] > -1, and why canceling a pending alarm early is done by explicitly setting it to -1, not to 0.